Dynamic Programming

Prof. Timothy Sun

Recap: fast exponentiation

Alg. based on **repeated squaring** $x^1, x^2, x^4, x^8, \ldots$

$\textsc{RecPow}(x, N)$:
    if $N = 0$
        return 1
    else if $N = 1$
        return x
    else
        return $\textsc{RecPow}(x, \lfloor N/2 \rfloor) \cdot \textsc{RecPow}(x, \lceil N/2 \rceil)$

Recap: fast exponentiation

Alg. based on **repeated squaring** $x^1, x^2, x^4, x^8, \ldots$

$\textsc{RecPow}(x, N)$:

if $N = 0$

return 1

else if $N = 1$

return x

else

return $\boxed{\textsc{RecPow}(x, \lfloor N/2 \rfloor) \cdot \textsc{RecPow}(x, \lceil N/2 \rceil)}$

This still runs in $\Theta(N)$ time.

Recap: fast exponentiation

Alg. based on **repeated squaring** $x^1, x^2, x^4, x^8, \ldots$

$\textsc{PowGood}(x, N)$:
    if $N = 0$
        return 1
    else
        $p := \textsc{PowGood}(x, \lfloor N/2 \rfloor)$
        if N is even: **return** $p \cdot p$
        else: **return** $p \cdot p \cdot x$

Recap: fast exponentiation

Alg. based on **repeated squaring** $x^1, x^2, x^4, x^8, \ldots$

$\textsc{PowGood}(x, N)$:
  if $N = 0$
    return 1
  else
    $p := \textsc{PowGood}(x, \lfloor N/2 \rfloor)$
    if N is even: **return** $p \cdot p$
    else: **return** $p \cdot p \cdot x$

Now runs in Θ(log ***N***) time (exponential speedup)

Avoiding repeated work

The **Fibonacci** numbers are the sequence

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

defined by the recurrence

$$F_1 = F_2 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

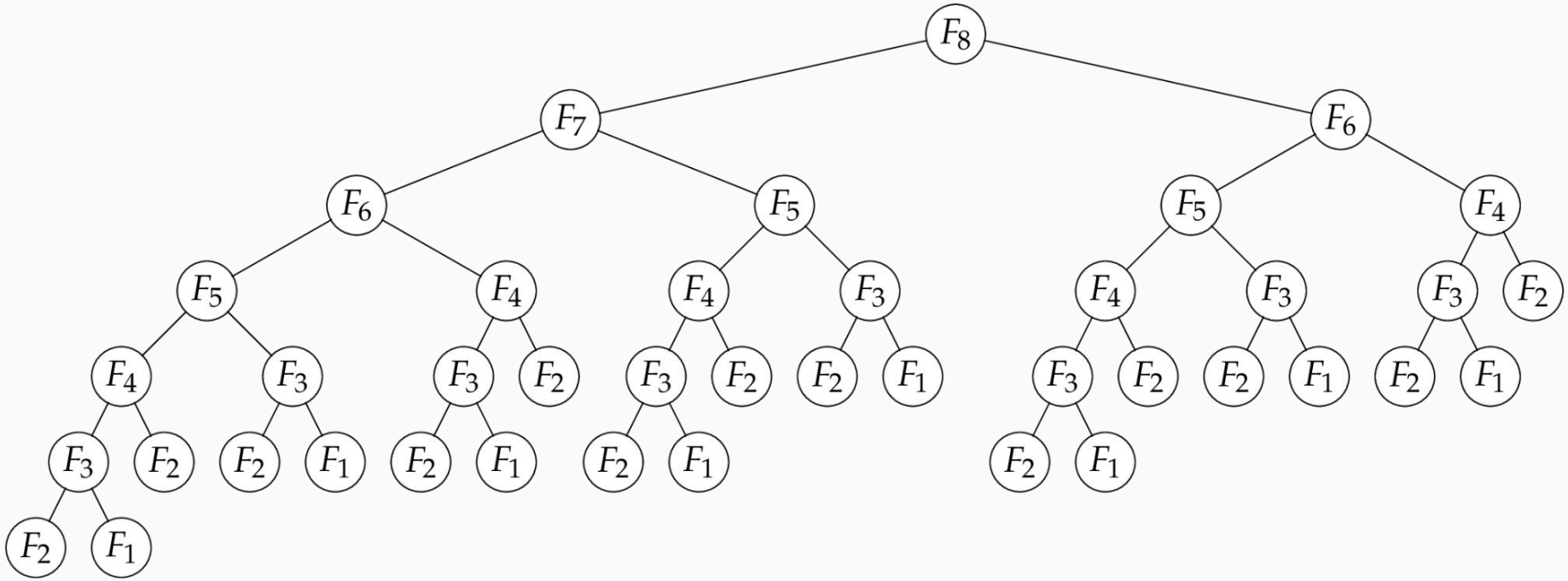
```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

Looks familiar...

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

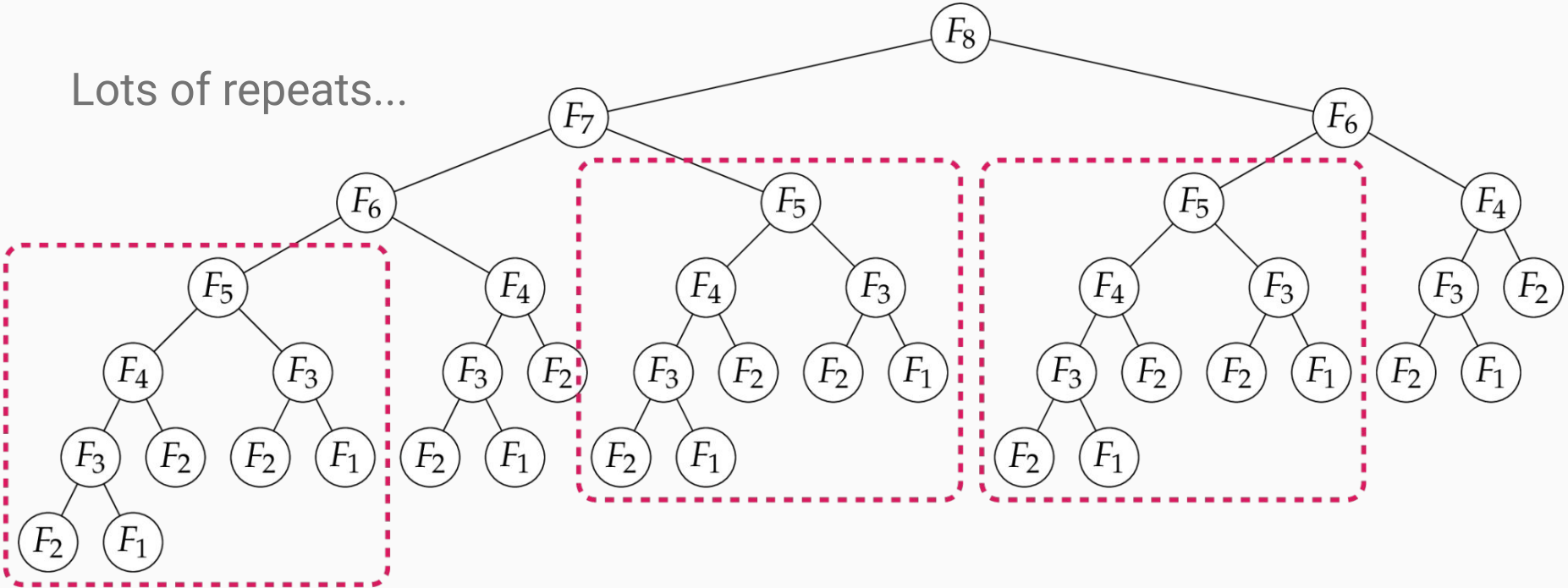
```
DOMINO(n):
    if n = 0,1
        return 1
    else
        return DOMINO(n − 1) + DOMINO(n − 2)
```

A recursion tree for Fibonacci numbers



A recursion tree for Fibonacci numbers

Lots of repeats...



Avoiding repeated work

The **Fibonacci** numbers are the sequence

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

defined by the recurrence

$$F_1 = F_2 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

Backtracking algorithms take exponential time!

Avoiding repeated work

What if we could **remember** what we computed before?

$$
\begin{aligned}
&\textsc{Fib}(n):\\
&\qquad \textbf{if } n = 1, 2\\
&\qquad\qquad \textbf{return } 1\\
&\qquad \textbf{else}\\
&\qquad\qquad \textbf{return } \textsc{Fib}(n-1) + \textsc{Fib}(n-2)
\end{aligned}
$$

Avoiding repeated work

What if we could **remember** what we computed before?

$F[1, \ldots]$
$\textsc{MFib}(n)$:
    if $F[n]$ is not defined
        if $n = 1, 2$
            $F[n] := 1$
        else
            $F[n] := \textsc{MFib}(n-1) + \textsc{MFib}(n-2)$
    return $F[n]$

Approach known as **memoization** [sic]... but what's the **runtime**?

Dynamic programming

Better way would be to fill up the table **without recursion**.

$$
\begin{aligned}
&\textsc{DPFib}(n):\\
&\qquad \text{initialize } F[1,\ldots,\max(2,n)]\\
&\qquad F[1] := 1\\
&\qquad F[2] := 1\\
&\qquad \textbf{for } i = 3,\ldots,n\\
&\qquad\qquad F[i] := F[i-1] + F[i-2]\\
&\qquad \textbf{return } F[n]
\end{aligned}
$$

Dynamic programming

Better way would be to fill up the table **without recursion**.

$\Theta(\boldsymbol{n})$ running time

$$
\begin{aligned}
&\textsc{DPFib}(n):\\
&\qquad \text{initialize } F[1,\ldots,\max(2,n)]\\
&\qquad F[1] := 1\\
&\qquad F[2] := 1\\
&\qquad \textbf{for } i = 3,\ldots,n\\
&\qquad\qquad F[i] := F[i-1] + F[i-2]\\
&\qquad \textbf{return } F[n]
\end{aligned}
$$

Dynamic programming

Better way would be to fill up the table **without recursion**.

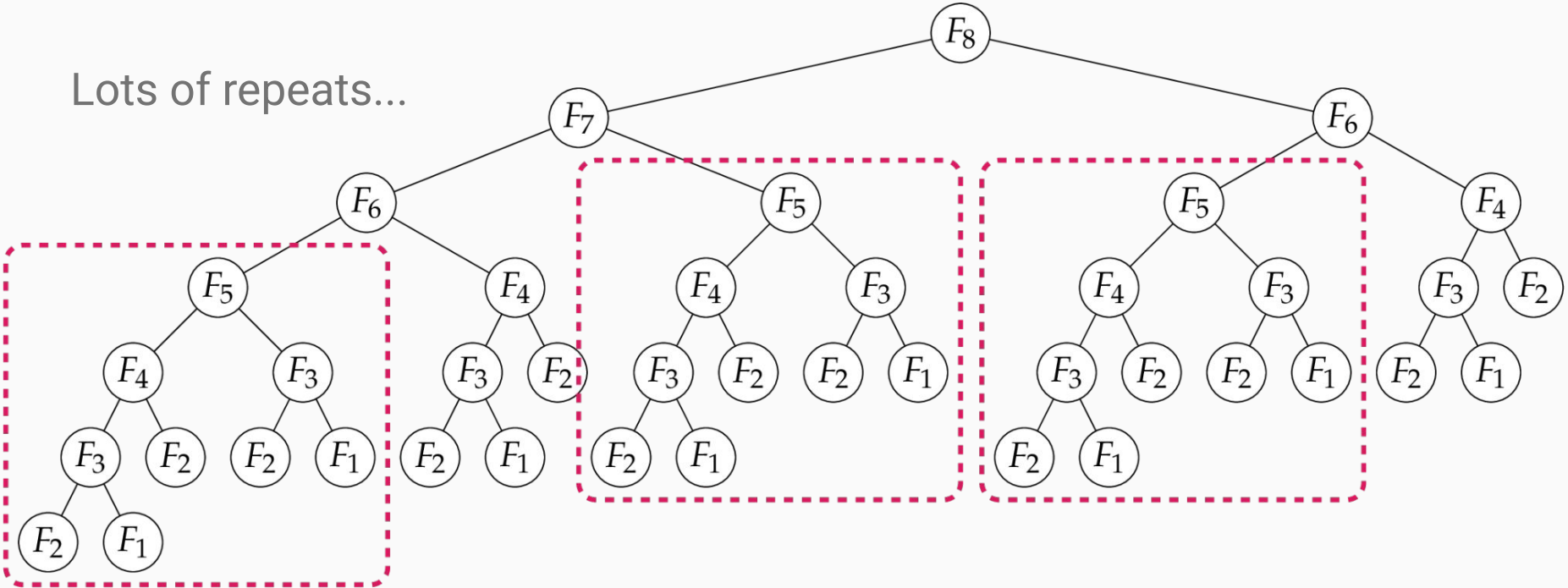
$\Theta(\boldsymbol{n})$ running time

Dynamic programming: solve **all** subproblems iteratively

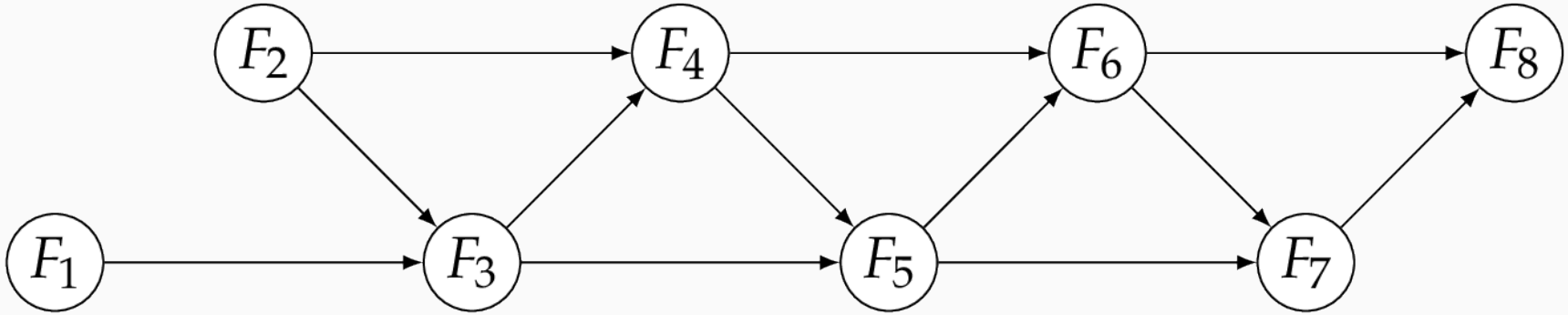
$$
\begin{aligned}
&\textsc{DPFib}(n):\\
&\qquad \text{initialize } F[1,\ldots,\max(2,n)]\\
&\qquad F[1] := 1\\
&\qquad F[2] := 1\\
&\qquad \textbf{for } i = 3,\ldots,n\\
&\qquad\qquad F[i] := F[i-1] + F[i-2]\\
&\qquad \textbf{return } F[n]
\end{aligned}
$$

A recursion tree for Fibonacci numbers

Lots of repeats...



Actual graph of dependencies



Much more compact...

Fibonacci Numbers

$\textsc{Fib}(n)$:
    if $n = 1, 2$
        return 1
    else
        return $\textsc{Fib}(n-1) + \textsc{Fib}(n-2)$

$\textsc{DPFib}(n)$:
    initialize $F[1, \ldots, \max(2, n)]$
    $F[1] := 1$
    $F[2] := 1$
    for $i = 3, \ldots, n$
        $F[i] := F[i-1] + F[i-2]$
    return $F[n]$

Backtracking vs. **Dynamic Programming**

Fibonacci Numbers

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

```
DPFIB(n):
    initialize F[1, ..., max(2, n)]
    F[1] := 1
    F[2] := 1
    for i = 3, ..., n
        F[i] := F[i − 1] + F[i − 2]
    return F[n]
```

Backtracking **vs.** **Dynamic Programming**

complex → simple

simple → complex

Fibonacci Numbers

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

```
DPFIB(n):
    initialize F[1, ..., max(2, n)]
    F[1] := 1
    F[2] := 1
    for i = 3, ..., n
        F[i] := F[i − 1] + F[i − 2]
    return F[n]
```

Backtracking **vs.** **Dynamic Programming**

Recursive calls replaced with array lookups

Dynamic Programming

Every* recurrence can be converted into a memoized or dynamic programming algorithm. (recurrences = pure functions)

Dynamic Programming

Every* recurrence can be converted into a memoized or dynamic programming algorithm. (recurrences = pure functions)

Only need to understand **how** (and not **why**) the backtracking algorithm works.

Components of Backtracking

1. Describe **subproblems**
2. Identify **base cases**
3. Derive a **recurrence**

```
FIB(n):
    if n = 1, 2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

Components of Backtracking

1. Describe **subproblems**

   Smaller Fibonacci numbers

2. Identify **base cases**

   F_1 and F_2

3. Derive a **recurrence**

   $F_n = F_{n-1} + F_{n-2}$

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

Components of DP

1. Creating a backtracking algorithm(!)
2. Storing subproblems
3. Determining a **fill order**

$\textsc{DPFib}(n)$:

$\quad$ initialize $F[1, \ldots, \max(2, n)]$

$\quad F[1] := 1$

$\quad F[2] := 1$

$\quad$ **for** $i = 3, \ldots, n$

$\quad\quad F[i] := F[i-1] + F[i-2]$

$\quad$ **return** $F[n]$

Components of DP

1. Creating a backtracking algorithm(!)

```
FIB(n):
    if n = 1,2
        return 1
    else
        return FIB(n − 1) + FIB(n − 2)
```

2. Storing subproblems

   Store in 1D array

3. Determining a **fill order**

   Increasing order

```
DPFIB(n):
    initialize F[1, ..., max(2, n)]
    F[1] := 1
    F[2] := 1
    for i = 3, ..., n
        F[i] := F[i − 1] + F[i − 2]
    return F[n]
```

Components of DP

Important things:

- Which subproblems are base cases?
- Which subproblems show up in the recurrence?

$\textsc{DPFib}(n)$:

$\quad$ initialize $F[1, \ldots, \max(2, n)]$

$\quad F[1] := 1$

$\quad F[2] := 1$

$\quad$ **for** $i = 3, \ldots, n$

$\quad\quad F[i] := F[i-1] + F[i-2]$

$\quad$ **return** $F[n]$

Components of DP

Important things:

- Which subproblems are base cases?
- Which subproblems show up in the recurrence?

Unimportant things:

- Why does the recurrence work?

$\textsc{DPFib}(n)$:

$\quad$ initialize $F[1, \ldots, \max(2, n)]$

$\quad F[1] := 1$

$\quad F[2] := 1$

$\quad$ **for** $i = 3, \ldots, n$

$\quad\quad F[i] := F[i-1] + F[i-2]$

$\quad$ **return** $F[n]$

Components of DP

If subproblems A, B, C are used in the recurrence to solve subproblem D, then A, B, C are **dependencies of** D.

$\textsc{DPFib}(n)$:

$\quad$ initialize $F[1, \ldots, \max(2, n)]$

$\quad F[1] := 1$

$\quad F[2] := 1$

$\quad$ **for** $i = 3, \ldots, n$

$\quad\quad F[i] := F[i-1] + F[i-2]$

$\quad$ **return** $F[n]$

Components of DP

If subproblems A, B, C are used in the recurrence to solve subproblem D, then A, B, C are **dependencies of** D.

Fill order: ordering of subproblems so that **dependencies are solved first**.

$\textsc{DPFib}(n)$:

$\quad$ initialize $F[1, \ldots, \max(2, n)]$

$\quad F[1] := 1$

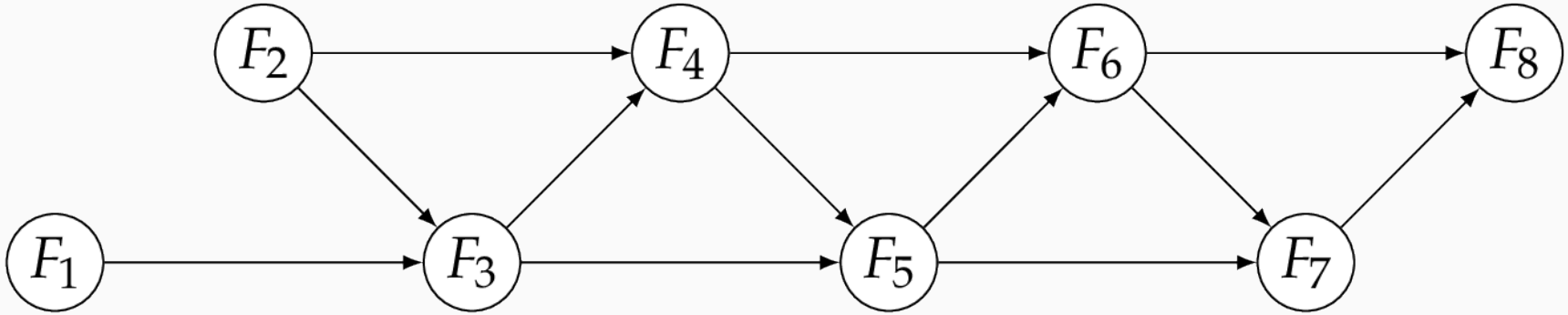
$\quad F[2] := 1$

$\quad$ **for** $i = 3, \ldots, n$

$\quad\quad F[i] := F[i-1] + F[i-2]$

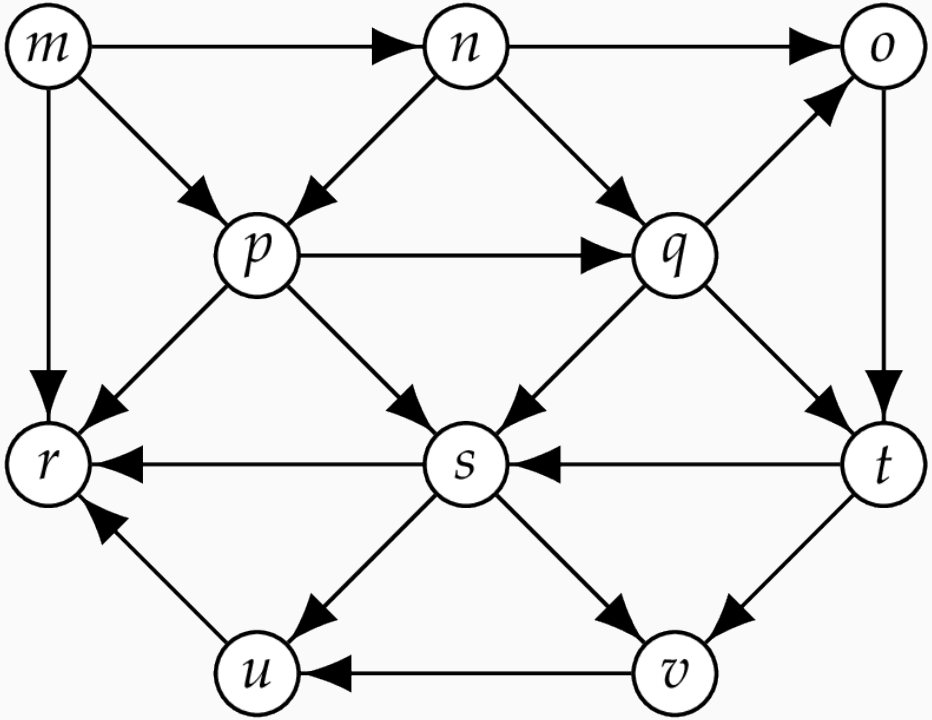
$\quad$ **return** $F[n]$

Actual graph of dependencies



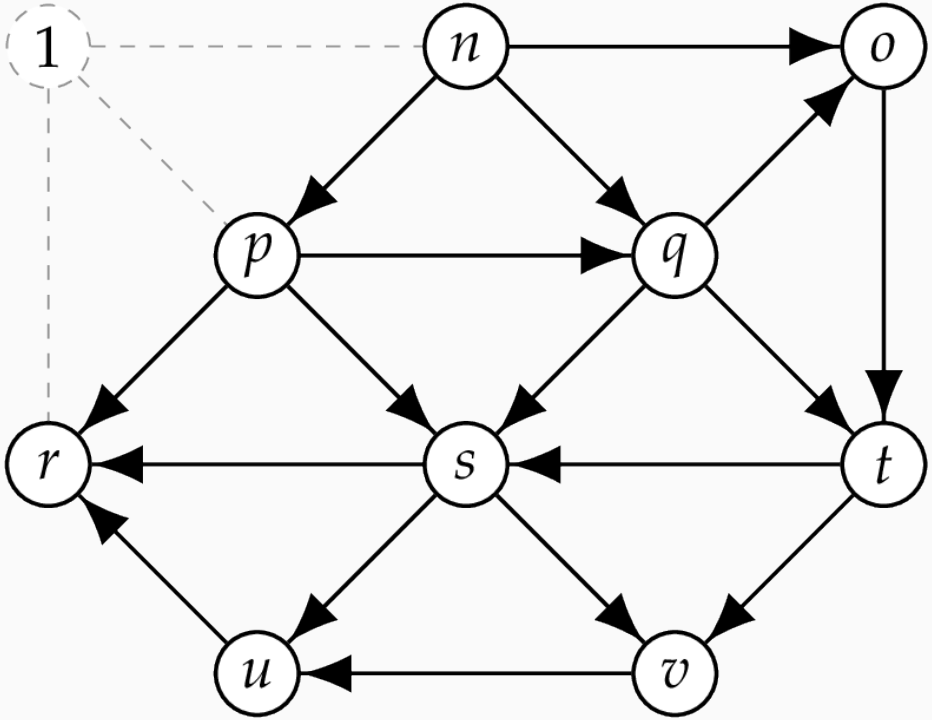
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



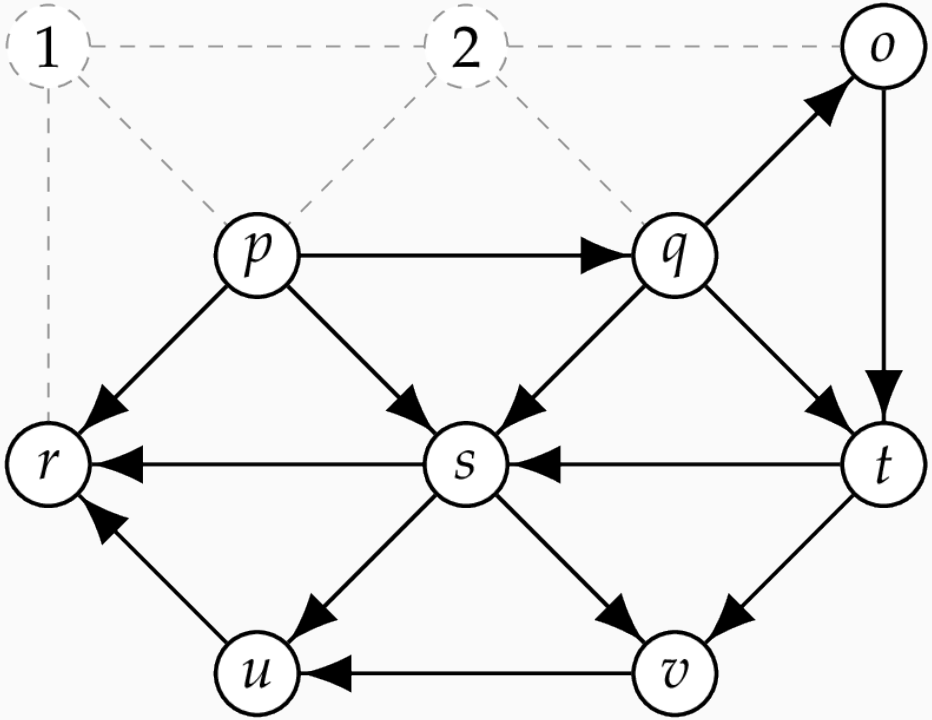
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



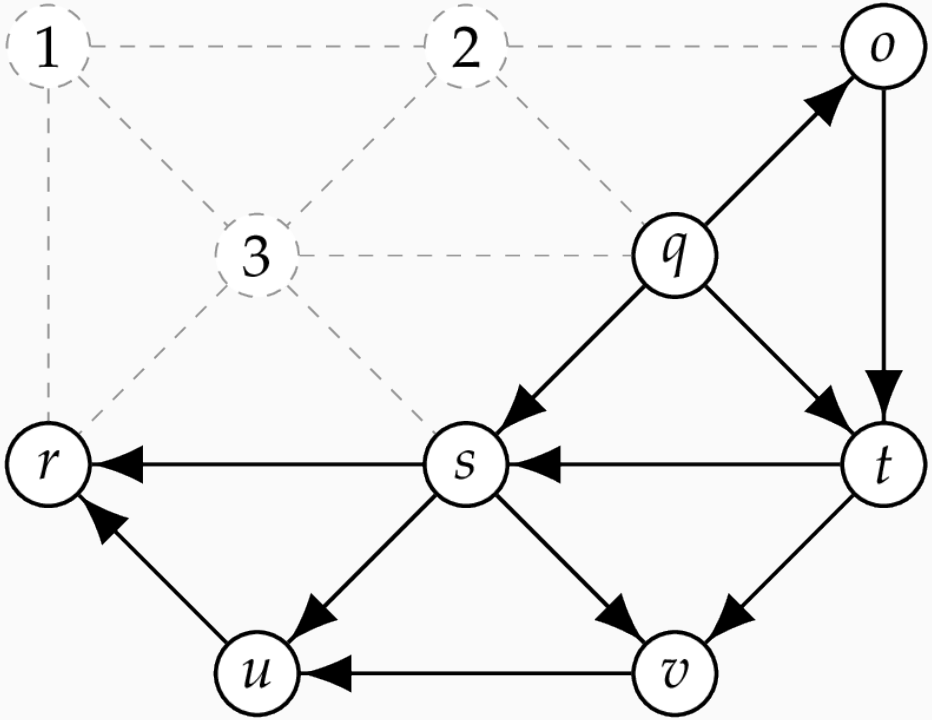
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



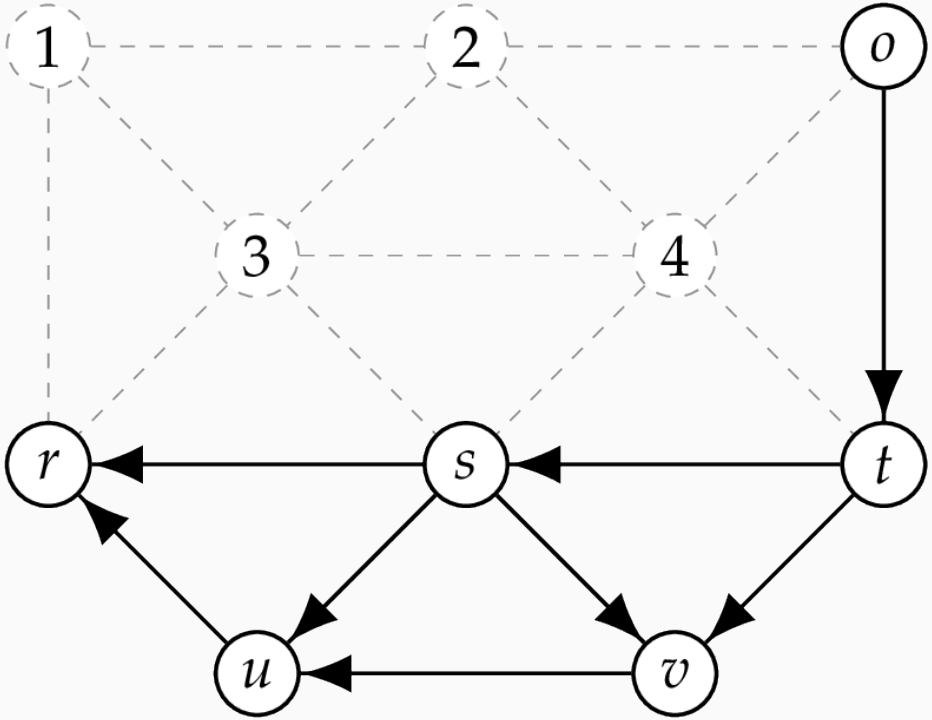
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



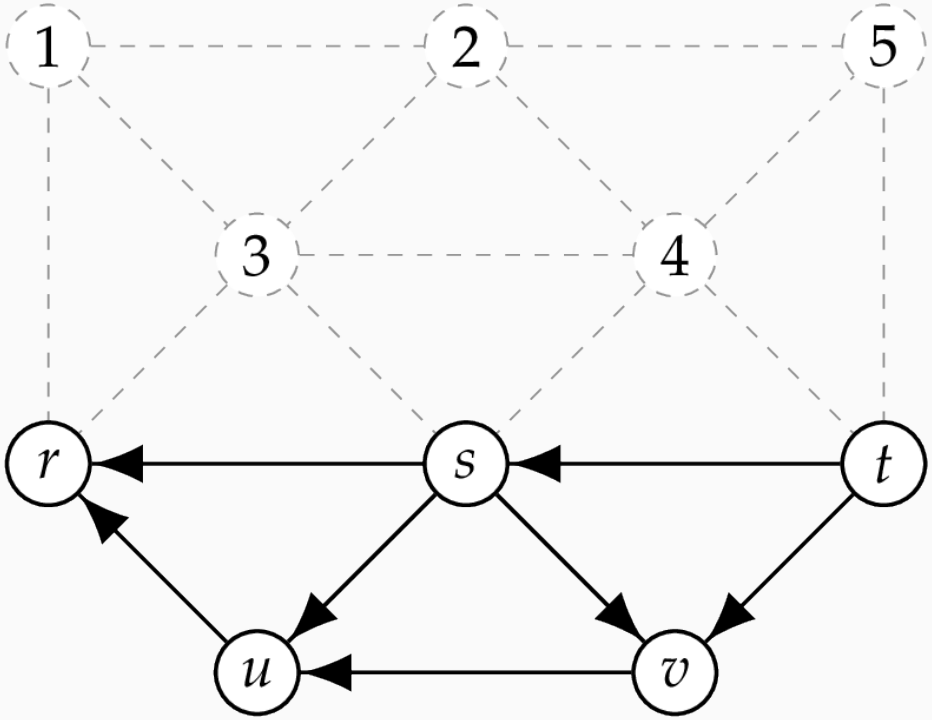
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



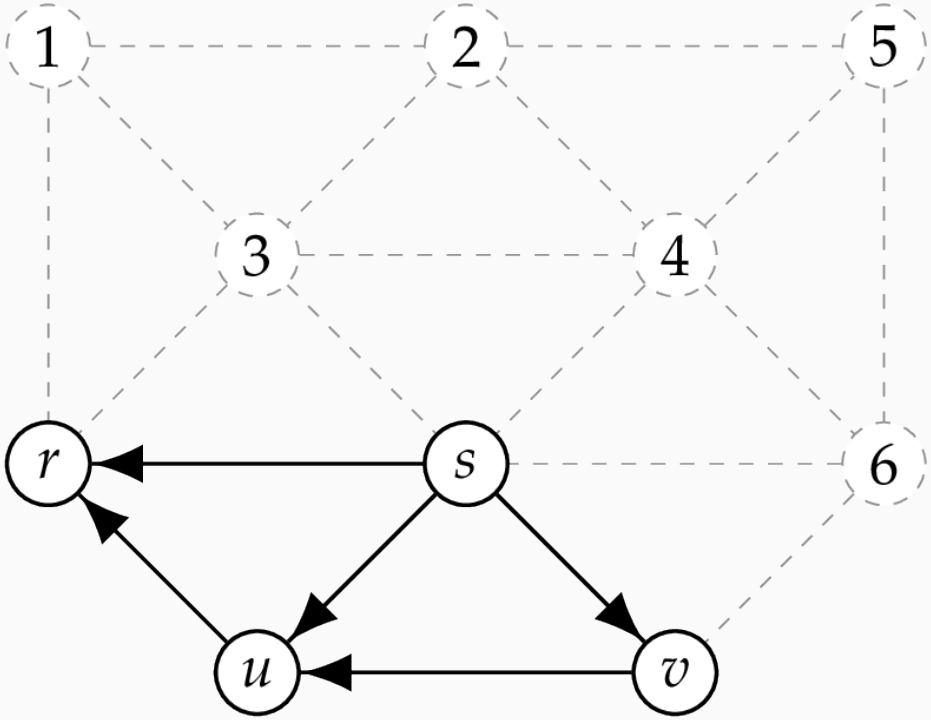
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



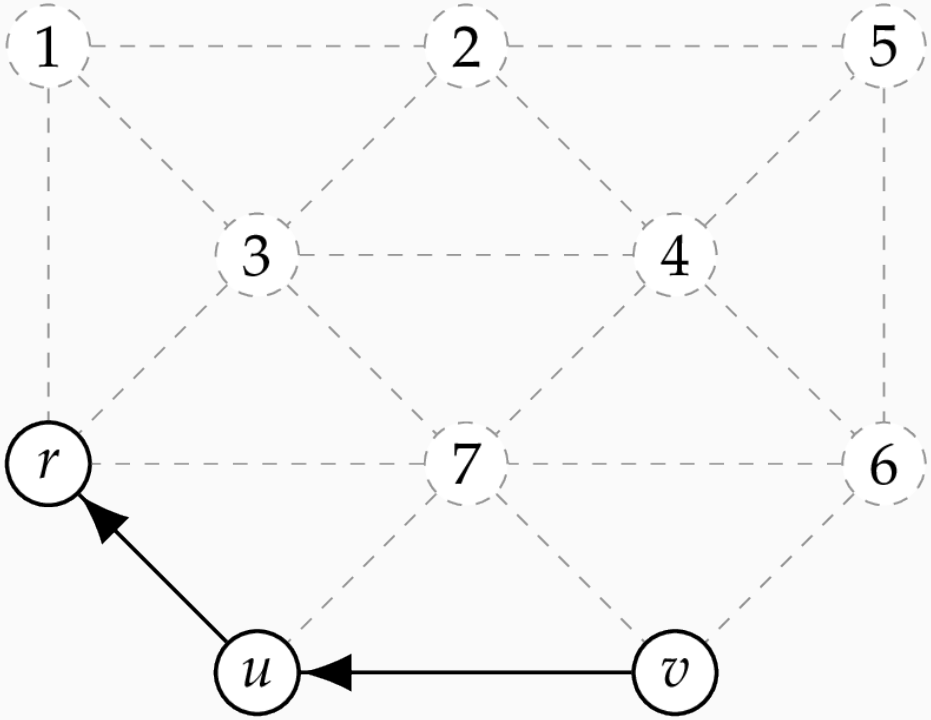
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



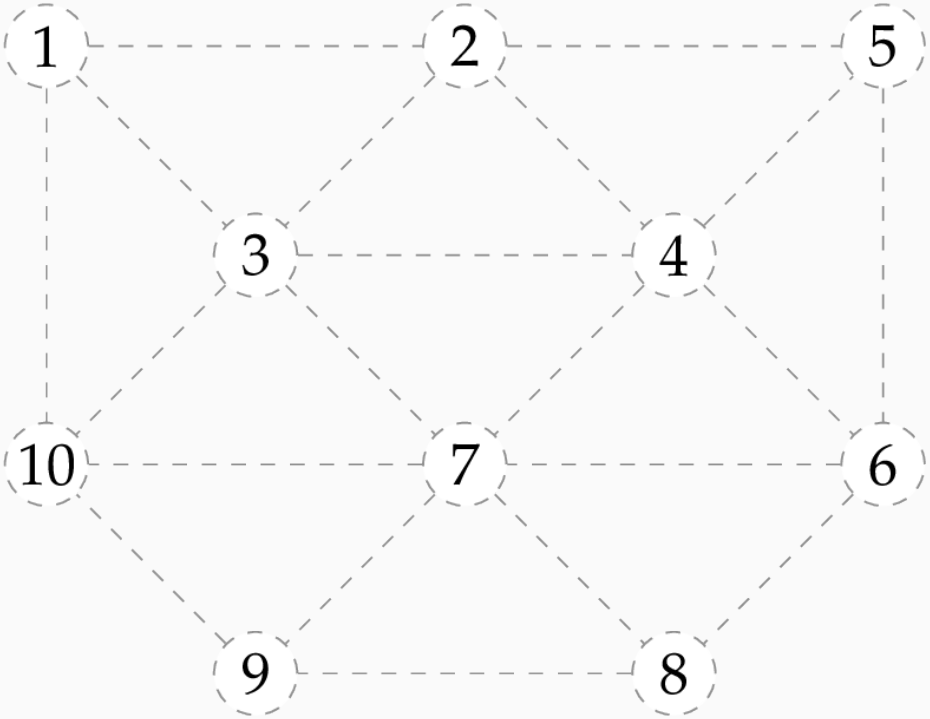
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



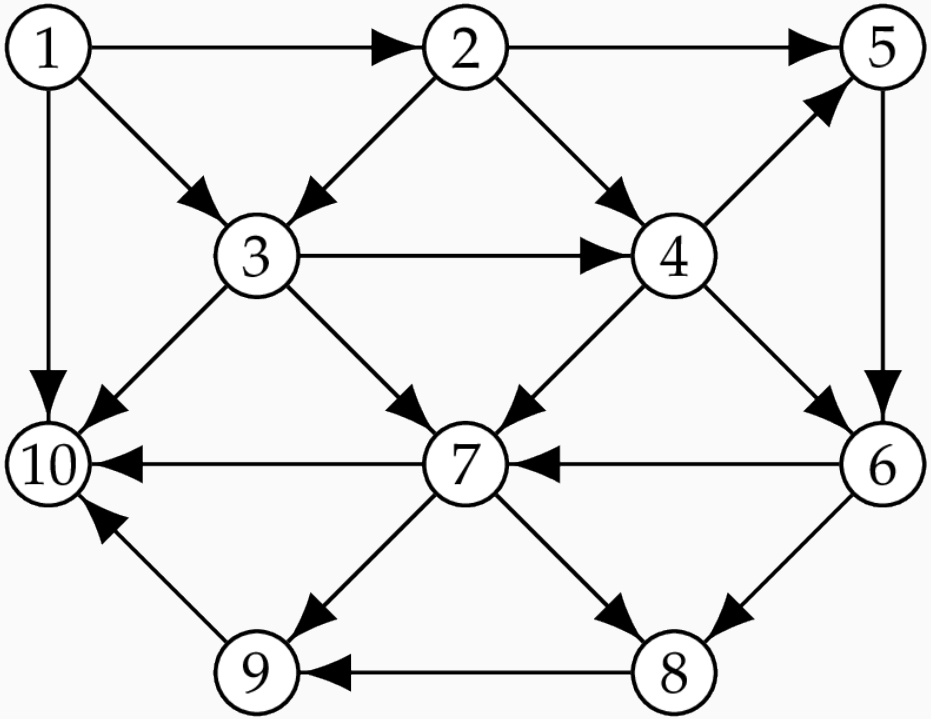
A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



A messy dependency graph

As long as there's no infinite recursion, a fill order is always possible:



Text splitting revisited

“Try every possible first word and recurse.”

```
T HISISTHEREASON

TH ISISTHEREASON

THI SISTHEREASON

...
```

Text splitting revisited

"Try every possible first word and recurse."

```
SPLITTABLE(S[1,...,n]):
    if n = 0
        return TRUE
    for i = 1,...,n
        if S[1,...,i] is a word
            if SPLITTABLE(S[i+1,...,n])
                return TRUE
    return FALSE
```

Text splitting revisited

"Try every possible first word and recurse." (version 2, indexed)

```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

Text splitting revisited

Subproblems: ***suffixes***, indexed by ***i***.

```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

Text splitting revisited

Subproblems: ***suffixes***, indexed by ***i***, can store in an array ***A***[1, …, ***n***+1]

```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

Text splitting revisited

In what order should the subproblems be solved?

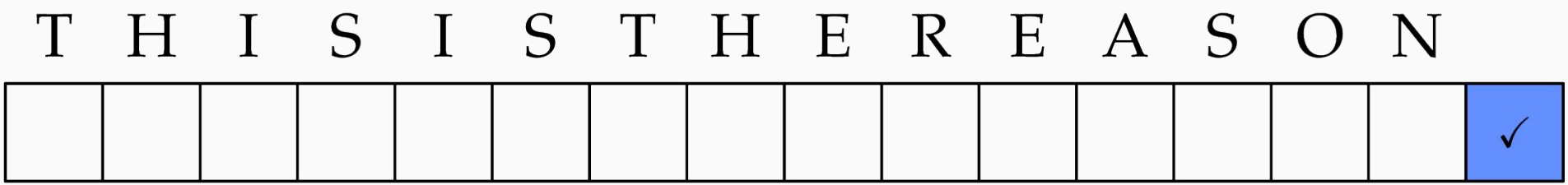
```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

Text splitting revisited

In what order should the subproblems be solved? ***Decreasing order***

```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

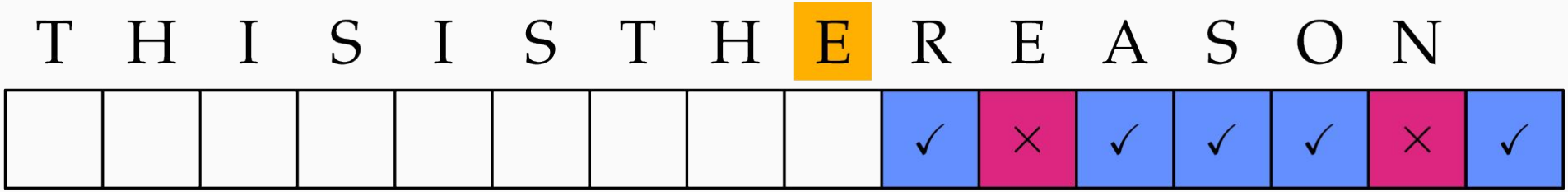
Text splitting revisited



Text splitting revisited

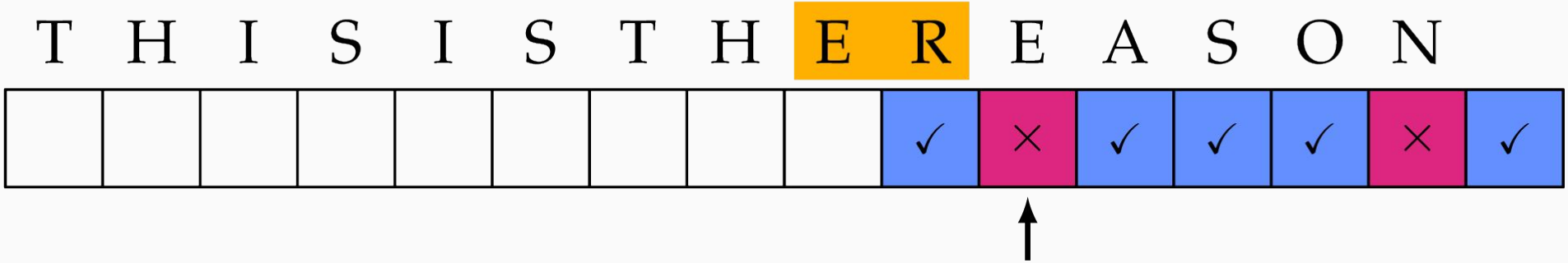
T	H	I	S	I	S	T	H	E	R	E	A	S	O	N	
									✓	×	✓	✓	✓	×	✓

Text splitting revisited



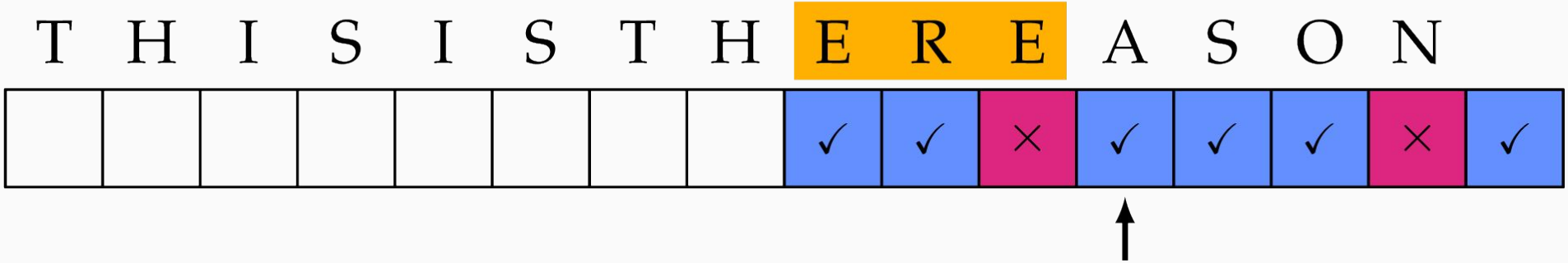
“E” is not a word, so let’s move on to the next prefix

Text splitting revisited



“ER” is a word (in Scrabble!), but the rest of the string isn’t splittable

Text splitting revisited

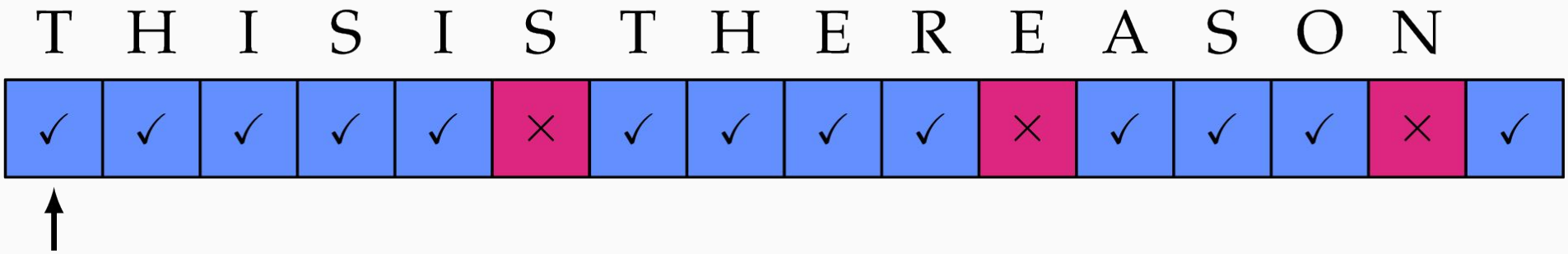


"ERE" is a word, ***and*** the rest of the string is splittable

Text splitting revisited

T	H	I	S	I	S	T	H	E	R	E	A	S	O	N	
✓	✓	✓	✓	✓	×	✓	✓	✓	✓	×	✓	✓	✓	×	✓

Text splitting revisited



The original string is splittable, as seen in the first cell.

The dynamic programming algorithm

```
DPSplit(S[1,...,n]):
    initialize A[1,...,n+1]
    A[n+1] := True
    for i = n,...,1
        A[i] := False
        for j = i,...,n
            if S[i,...,j] is a word
                if A[j+1]
                    A[i] := True
    return A[1]
```

The dynamic programming algorithm

```
DPSplit(S[1,...,n]):
    initialize A[1,...,n+1]
    A[n+1] := True
    for i = n,...,1
        A[i] := False
        for j = i,...,n
            if S[i,...,j] is a word
                if A[j+1]
                    A[i] := True
    return A[1]
```

Runtime: $\Theta(\boldsymbol{n}^2)$

Generic DP structure

```
GENERICDYNAMIC(inputs):
    // create table for storing subproblems
    // fill in base cases
    for subproblem in fill order
        // solve subproblem using table lookups
        // store solution in table
    // return table value for original problem
```

Generic DP structure

Convert recurrence or backtracking algorithm here

```
GENERICDYNAMIC(inputs):
    // create table for storing subproblems
    // fill in base cases
    for subproblem in fill order
        // solve subproblem using table lookups
        // store solution in table
    // return table value for original problem
```

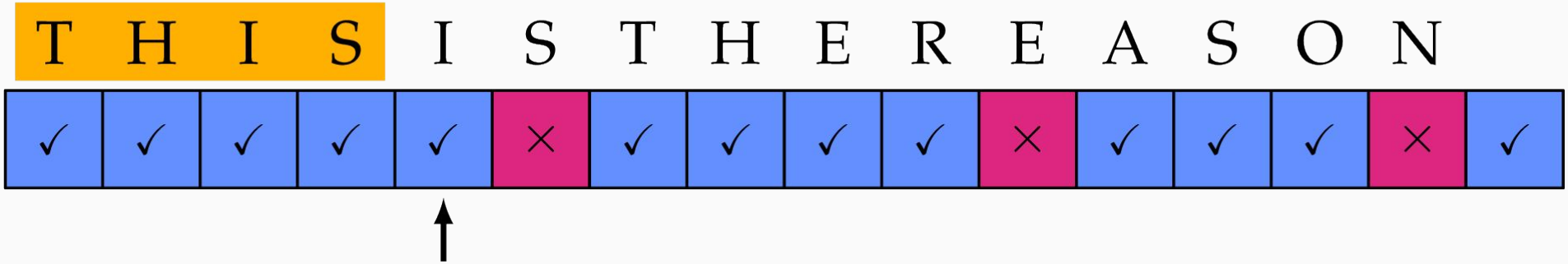
Reconstruction

Our algorithm only return **true** or **false**. How to find the actual splitting?

T	H	I	S	I	S	T	H	E	R	E	A	S	O	N	
✓	✓	✓	✓	✓	×	✓	✓	✓	✓	×	✓	✓	✓	×	✓

Reconstruction

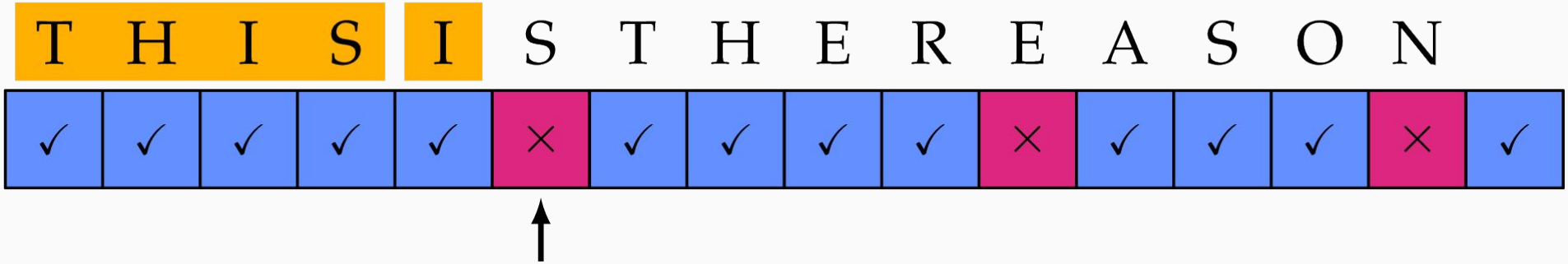
Our algorithm only return **true** or **false**. How to find the actual splitting?



Easy to figure out which first word is correct.

Reconstruction

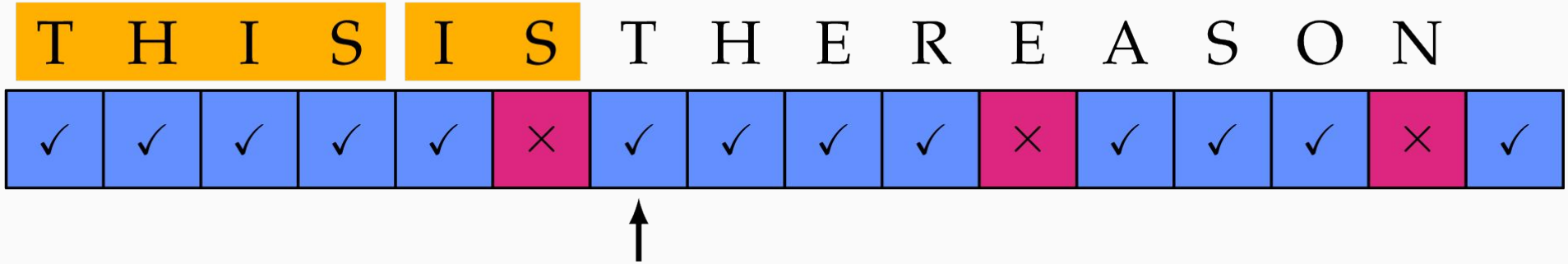
Our algorithm only return **true** or **false**. How to find the actual splitting?



Easy to figure out which first word is correct.

Reconstruction

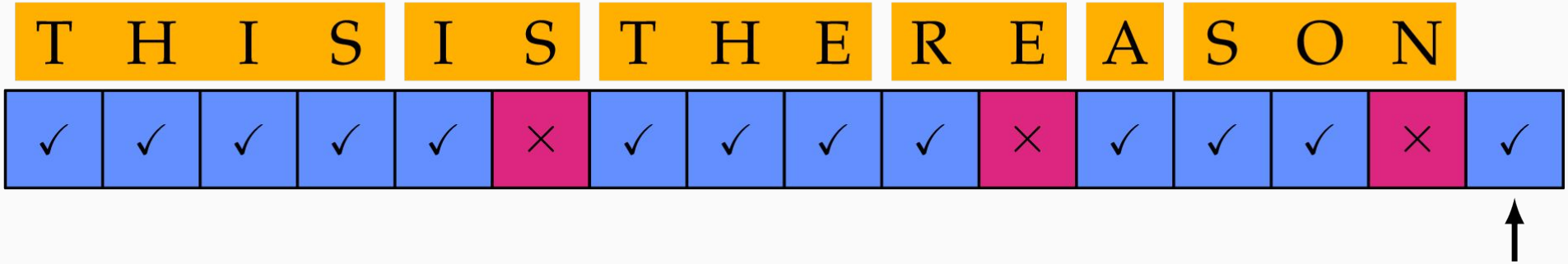
Our algorithm only return **true** or **false**. How to find the actual splitting?



Easy to figure out which first word is correct.

Reconstruction

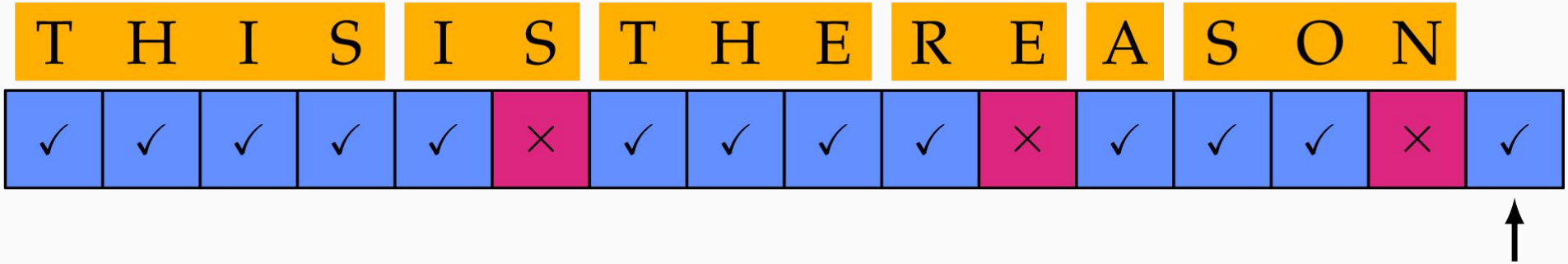
Our algorithm only return **true** or **false**. How to find the actual splitting?



Easy to figure out which first word is correct.

Reconstruction

Our algorithm only return **true** or **false**. How to find the actual splitting?



Opposite direction from dynamic programming
(Same direction as backtracking)

Reconstruction

```
SPLITTABLE(S[1,...,n], i):
    if i = n + 1
        return TRUE
    for j = i,...,n
        if S[i,...,j] is a word
            if SPLITTABLE(S, j + 1)
                return TRUE
    return FALSE
```

Essentially backtracking, but already knowing the right path.

Reconstruction

```
SplittableReconstruct(S[1,...,n]):
    // fill in the table A[1,...,n+1]
    if A[1]
        i := 1
        while i < n+1
            for j = i,...,n
                if S[i,...,j] is a word and A[j+1]
                    print S[i,...,j]
                    i := j+1
                    break
```

When is DP useful?

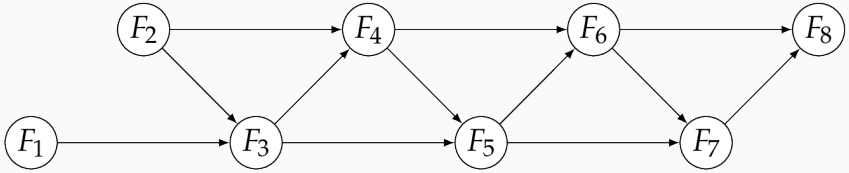
Backtracking takes exponential time

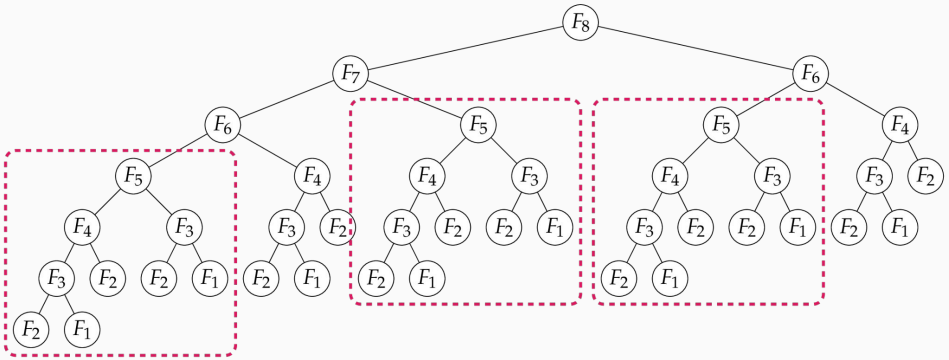
With **overlap**, the number of subproblems might be much smaller

When is DP useful?

Backtracking takes exponential time

With **overlap**, the number of subproblems might be much smaller



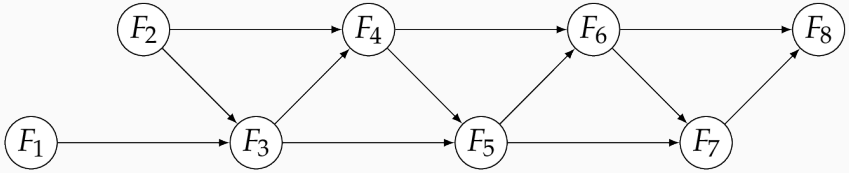


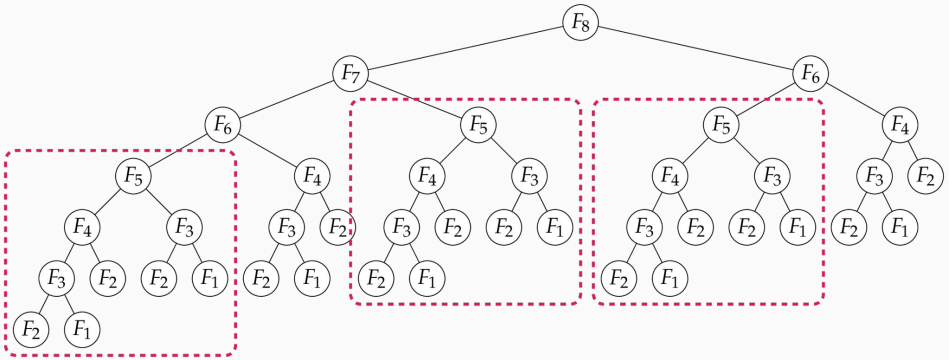
When is DP useful?

Backtracking takes exponential time

With **overlap**, the number of subproblems might be much smaller

- polynomial # of subproblems → polynomial time DP





Subset Sum revisited

Is there a subset of ***S*** that adds up to ***T***?

$$
\begin{aligned}
&\textsc{SubsetSum}(S[1,\ldots,n], \boxed{i,t}):\\
&\qquad \textbf{if } t = 0\\
&\qquad\qquad \textbf{return } \textsc{True}\\
&\qquad \textbf{else if } t < 0 \text{ or } i = 0\\
&\qquad\qquad \textbf{return } \textsc{False}\\
&\qquad \textbf{else}\\
&\qquad\qquad e := S[i]\\
&\qquad\qquad \mathit{include} := \textsc{SubsetSum}(S, i-1, t-e)\\
&\qquad\qquad \mathit{exclude} := \textsc{SubsetSum}(S, i-1, t)\\
&\qquad\qquad \textbf{return } \mathit{include} \text{ or } \mathit{exclude}
\end{aligned}
$$

Dependencies in Subset Sum

$\textsc{SubsetSum}(S[1,\ldots,n], i, t)$:

// some stuff

$\quad$ *include* := $\textsc{SubsetSum}(S, i-1, t-e)$

$\quad$ *exclude* := $\textsc{SubsetSum}(S, i-1, t)$

$\quad$ **return** *include* or *exclude*

(we'll deal with negative ***t*** later)

$t \rightarrow$

$i \downarrow$

0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8
1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8
2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8

Dependencies in Subset Sum

$\textsc{SubsetSum}(S[1,\ldots,n], i, t)$:

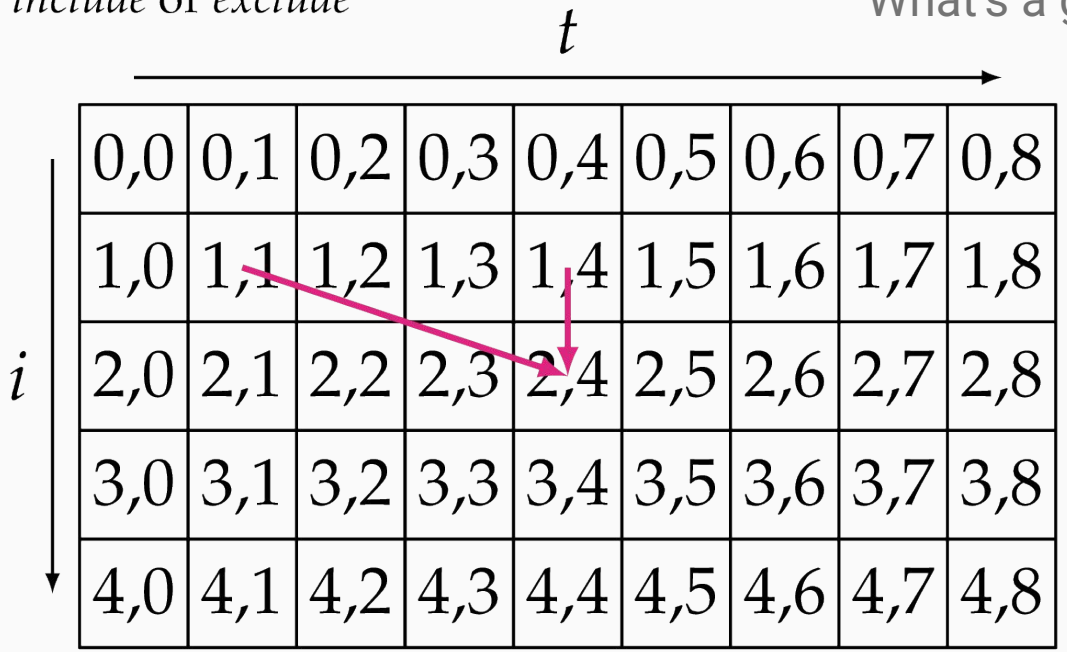
// some stuff

$\quad$ *include* := $\textsc{SubsetSum}(S, i-1, t-e)$

$\quad$ *exclude* := $\textsc{SubsetSum}(S, i-1, t)$

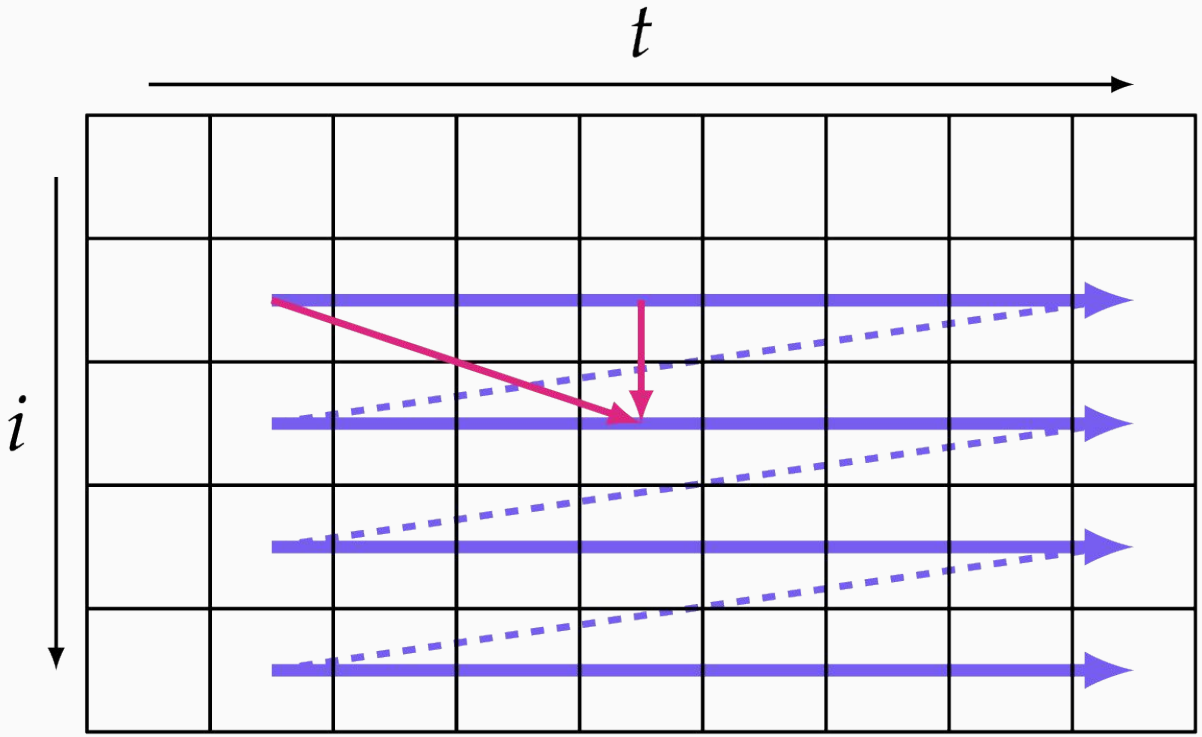
$\quad$ **return** *include* or *exclude*

What's a good **fill order**?



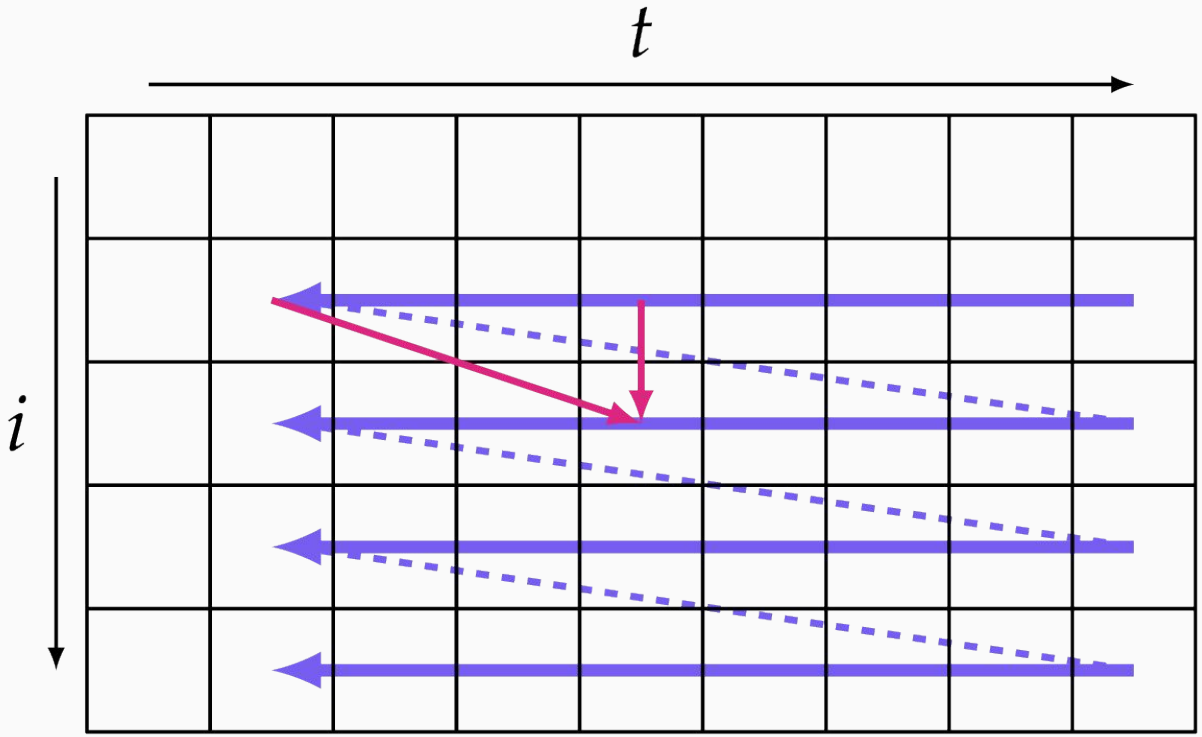
Fill order

Usually, row-by-row or column-by-column works



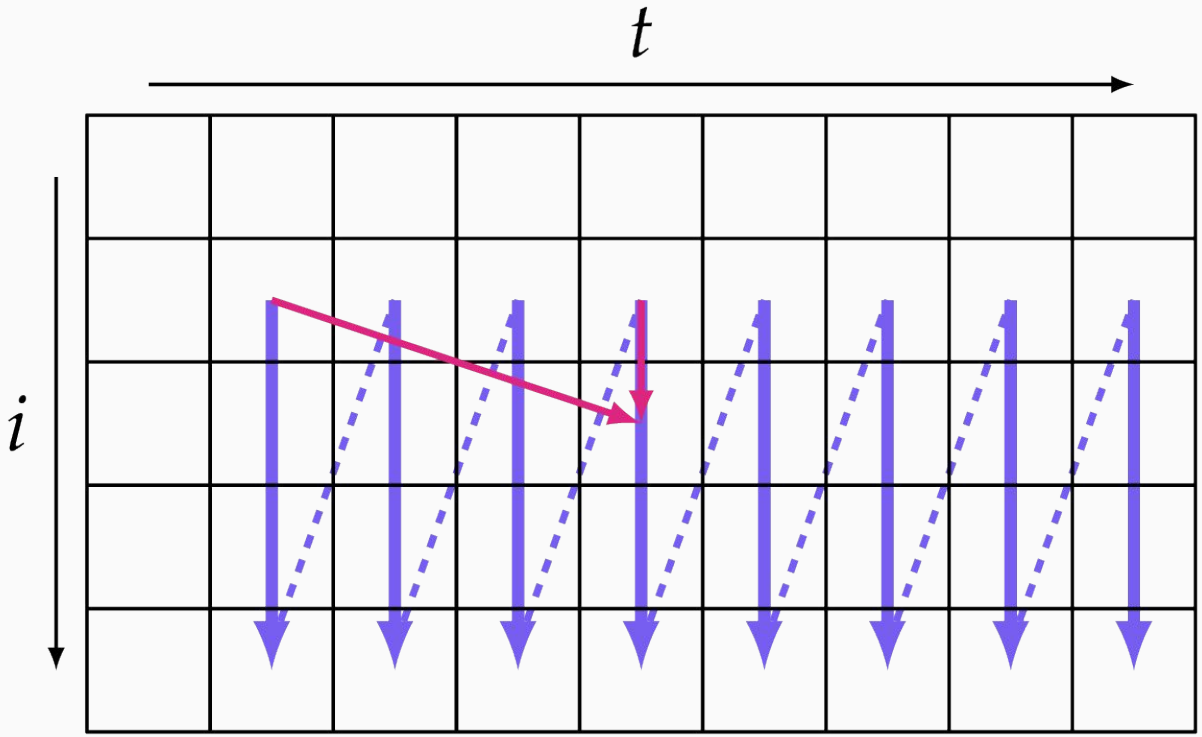
Fill order

Usually, row-by-row or column-by-column works



Fill order

Usually, row-by-row or column-by-column works

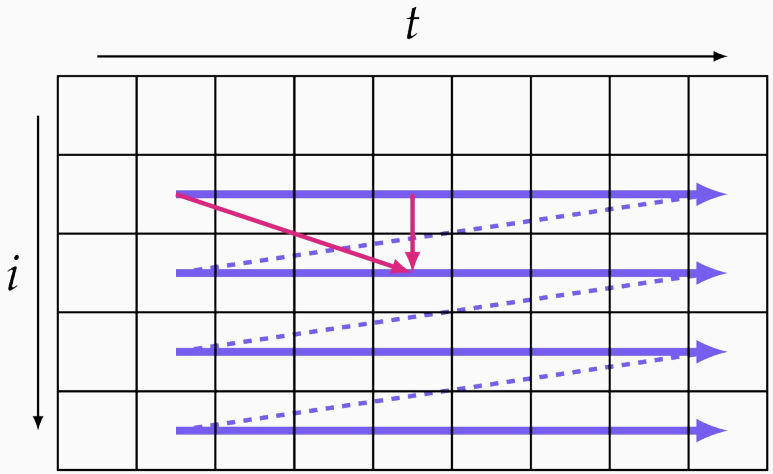


Subset Sum revisited

Storing subproblems: 2D array, ***i*** = 0, …, ***n***, ***t*** = 0, …, ***T***

Dependencies: (***i, t***) depends on (***i***-1, ***t***) and (***i***-1, ***t***-***S***[***i***])

Fill order: e.g., increasing ***i***, <u>doesn't matter</u> for ***T***



Subset Sum revisited

What does the 2D array look like for:

S = [1, 4, 3, 2], ***T*** = 7?

Subset Sum revisited

What does the 2D array look like for:

S = [1, 4, 3, 2], ***T*** = 7?

	0	1	2	3	4	5	6	7
	✓							
1	✓							
4	✓							
3	✓							
2	✓							

Subset Sum revisited

What does the 2D array look like for:

S = [1, 4, 3, 2], ***T*** = 7?

	0	1	2	3	4	5	6	7
	✓							
1	✓	✓						
4	✓	✓			✓	✓		
3	✓	✓		✓	✓	✓		✓
2	✓	✓	✓	✓	✓	✓	✓	✓

Subset Sum revisited

What does the 2D array look like for:

S = [1, 4, 3, 2], ***T*** = 7?

There is a solution, since the bottom-right cell is TRUE

	0	1	2	3	4	5	6	7
	✓							
1	✓	✓						
4	✓	✓			✓	✓		
3	✓	✓		✓	✓	✓		✓
2	✓	✓	✓	✓	✓	✓	✓	✓

Subset Sum revisited

S = [5, 2, 4, 7, 11], ***T*** = 19?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	✓																			
5	✓					✓														
2	✓		✓			✓		✓												
4	✓		✓		✓	✓	✓	✓		✓		✓								
7	✓		✓		✓	✓	✓	✓		✓		✓	✓	✓	✓		✓		✓	
11	✓		✓		✓	✓	✓	✓		✓		✓	✓	✓	✓	✓	✓	✓	✓	

Subset Sum revisited

S = [5, 2, 4, 7, 11], ***T*** = 19?

No solution

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	✓																			
5	✓					✓														
2	✓		✓			✓		✓												
4	✓		✓		✓	✓	✓	✓		✓		✓								
7	✓		✓		✓	✓	✓	✓		✓		✓	✓	✓	✓		✓		✓	
11	✓		✓		✓	✓	✓	✓		✓		✓	✓	✓	✓	✓	✓	✓	✓	

Subset Sum - dynamic

```
SubsetSumDP(S[1,...,n], T):
    initialize 2D array A[0,...,n][0,...,T]
    for i = 0,...,n
        A[i][0] := True
    for t = 1,...,T
        A[0][t] := False
    for i = 1,...,n
        for t = 1,...,S[i] − 1
            A[i][t] := A[i − 1][t]
        for t = S[i],...,T
            A[i][t] := A[i − 1][t − S[i]] or A[i − 1][t]
    return A[n][T]
```

Fleshed out

$\textsc{SubsetSumDP}(S[1,\ldots,n], T)$:

initialize 2D array $A[0,\ldots,n][0,\ldots,T]$

for $i = 0,\ldots,n$

$A[i][0] := \textsc{True}$

for $t = 1,\ldots,T$

$A[0][t] := \textsc{False}$

for $i = 1,\ldots,n$

$e := S[i]$

$exclude := A[i-1][t]$

for $t = 1,\ldots,S[i]-1$

$A[i][t] := exclude$

for $t = S[i],\ldots,T$

$include := A[i-1][t-e]$

$A[i][t] := include$ or $exclude$

return $A[n][T]$

To avoid negative ***t***, split the loop into two cases

Subset Sum revisited

Runtime: how long did it take to fill in the table?

- Usually O(# dependencies * # entries)

Subset Sum revisited

Runtime: how long did it take to fill in the table?

- Usually O(# dependencies * # entries)

Subset Sum: O(***nT***)

Subset Sum revisited

Runtime: how long did it take to fill in the table?

- Usually O(# dependencies * # entries)

Subset Sum: O(***nT***) - does this mean P = NP???

Subset Sum revisited

Runtime: how long did it take to fill in the table?

- Usually O(# dependencies * # entries)

Subset Sum: O(***nT***) - does this mean P = NP???

No, ***T*** is exponential in number of digits

Wood cutting

length	1	2	3	4	5	6	7	8	9	10
price	\$1	\$5	\$8	\$9	\$10	\$17	\$17	\$20	\$24	\$30

$\$10$

$\$5 + \$8 = \$13$

The backtracking

$\textsc{WoodCut}(p[1,\ldots,N], m)$:

$\quad best := 0$

$\quad \textbf{for } i = 1,\ldots,m$

$\qquad profit := p[i] + \textsc{WoodCut}(p, m-i)$

$\qquad \textbf{if } profit > best$

$\qquad\qquad best := profit$

$\quad \textbf{return } best$

$$W(m) = \begin{cases} 0 & \text{if } m = 0 \\ \max_{i=1}^{m} p[i] + W(m-i) & \text{otherwise} \end{cases}$$

The backtracking

$$
\begin{aligned}
&\textsc{WoodCut}(p[1,\ldots,N], m):\\
&\qquad best := 0\\
&\qquad \textbf{for } i = 1,\ldots,m\\
&\qquad\qquad profit := p[i] + \textsc{WoodCut}(p, m-i)\\
&\qquad\qquad \textbf{if } profit > best\\
&\qquad\qquad\qquad best := profit\\
&\qquad \textbf{return } best
\end{aligned}
$$

Storing subproblems?

Fill order?

The backtracking

$$
\begin{aligned}
&\textsc{WoodCut}(p[1,\ldots,N], m):\\
&\qquad best := 0\\
&\qquad \textbf{for } i = 1,\ldots,m\\
&\qquad\qquad profit := p[i] + \textsc{WoodCut}(p, m-i)\\
&\qquad\qquad \textbf{if } profit > best\\
&\qquad\qquad\qquad best := profit\\
&\qquad \textbf{return } best
\end{aligned}
$$

Storing subproblems: 1D array [0, …, ***M***]

Fill order?

The backtracking

$$
\begin{aligned}
&\textsc{WoodCut}(p[1,\ldots,N], m):\\
&\qquad best := 0\\
&\qquad \textbf{for } i = 1,\ldots,m\\
&\qquad\qquad profit := p[i] + \textsc{WoodCut}(p, m-i)\\
&\qquad\qquad \textbf{if } profit > best\\
&\qquad\qquad\qquad best := profit\\
&\qquad \textbf{return } best
\end{aligned}
$$

Storing subproblems: 1D array [0, …, ***M***]

Fill order: increasing order

Wood cutting

```
WoodCutDP(p[1,...,N], M):
    initialize array R[0,...,M]
    R[0] := 0
    for m = 1,...,M
        best := 0
        for i = 1,...,m
            profit := p[i] + R[m − i]
            if profit > best
                best := profit
        R[m] := best
    return R[M]
```

Wood cutting

```
WoodCutDP(p[1,...,N], M):
    initialize array R[0,...,M]
    R[0] := 0
    for m = 1,...,M
        best := 0
        for i = 1,...,m
            profit := p[i] + R[m − i]
            if profit > best
                best := profit
        R[m] := best
    return R[M]
```

Runtime: $\Theta(M^2)$